

MySQL8新特性底层原理

降序索引

什么是降序索引

MySQL 8.0开始真正支持降序索引 (descending index)。只有InnoDB存储引擎支持降序索引，只支持B+TREE降序索引。另外MySQL8.0不再对GROUP BY操作进行隐式排序。

在MySQL中创建一个t2表

```
create table t2(c1 int,c2 int,index idx1(c1 asc,c2 desc));

show create table t2\G
```

```
mysql> show create table t2\G
***** 1. row *****
Table: t2
Create Table: CREATE TABLE `t2` (
  `c1` int DEFAULT NULL,
  `c2` int DEFAULT NULL,
  KEY `idx1` (`c1`,`c2` DESC)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci
1 row in set (0.00 sec)
```

如果是5.7中，则没有显示升序还是降序信息

```
mysql> create table t2(c1 int,c2 int,index idx1(c1 asc,c2 desc));
Query OK, 0 rows affected (0.02 sec)

mysql> show create table t2\G
***** 1. row *****
Table: t2
Create Table: CREATE TABLE `t2` (
  `c1` int(11) DEFAULT NULL,
  `c2` int(11) DEFAULT NULL,
  KEY `idx1` (`c1`,`c2`)
) ENGINE=InnoDB DEFAULT CHARSET=latin1
1 row in set (0.00 sec)
```

我们插入一些数据，给大家演示下降序索引的使用

```
insert into t2(c1,c2) values(1,100),(2,200),(3,150),(4,50);
```

```
mysql> select * from t2;
```

c1	c2
1	100
2	200
3	150
4	50

看下索引使用情况

```
explain select * from t2 order by c1,c2 desc;
```

```
mysql> explain select * from t2 order by c1,c2 desc;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t2	NULL	index	NULL	idx1	10	NULL	4	100.00	Using index

1 row in set, 1 warning (0.00 sec)

我们在5.7对比一下

```
mysql> explain select * from t2 order by c1,c2 desc;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t2	NULL	index	NULL	idx1	10	NULL	4	100.00	Using index; Using filesort

1 row in set, 1 warning (0.00 sec)

这里说明，这里需要一个额外的排序操作，才能把刚才的索引利用上。

我们把查询语句换一下

```
explain select * from t2 order by c1 desc,c2 ;
```

MySQL8中使用了

```
mysql> explain select * from t2 order by c1 desc,c2 ;
```

id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	SIMPLE	t2	NULL	index	NULL	idx1	10	NULL	4	100.00	Backward index scan; Using index

1 row in set, 1 warning (0.00 sec)

另外还有一点，就是group by语句在 8之后不再默认排序

```
select count(*),c2 from t2 group by c2;
```

```
mysql> select count(*),c2 from t2 group by c2;
```

count(*)	c2
1	100
1	200
1	150
1	50

4 rows in set (0.00 sec)

MySQL8

```
mysql> select count(*),c2 from t2 group by c2;
```

count(*)	c2
1	50
1	100
1	150
1	200

MySQL5.7

在8要排序的话，就需要手动把排序语句加上

```
select count(*),c2 from t2 group by c2 order by c2;
```

```
mysql> select count(*),c2 from t2 group by c2 order by c2;
```

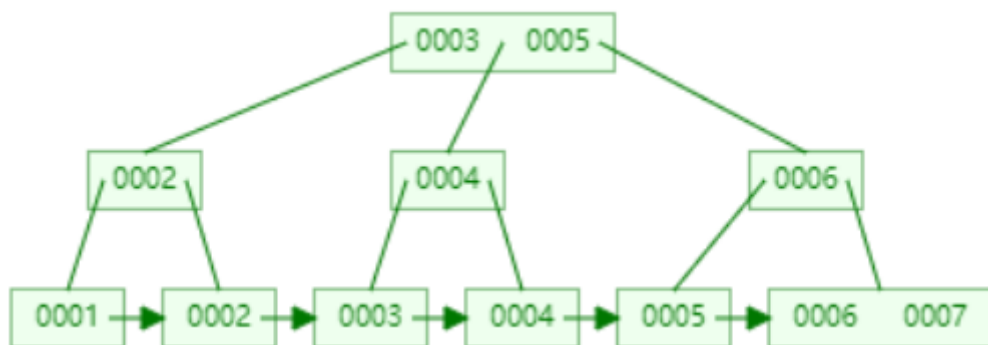
count(*)	c2
1	50
1	100
1	150
1	200

4 rows in set (0.00 sec)

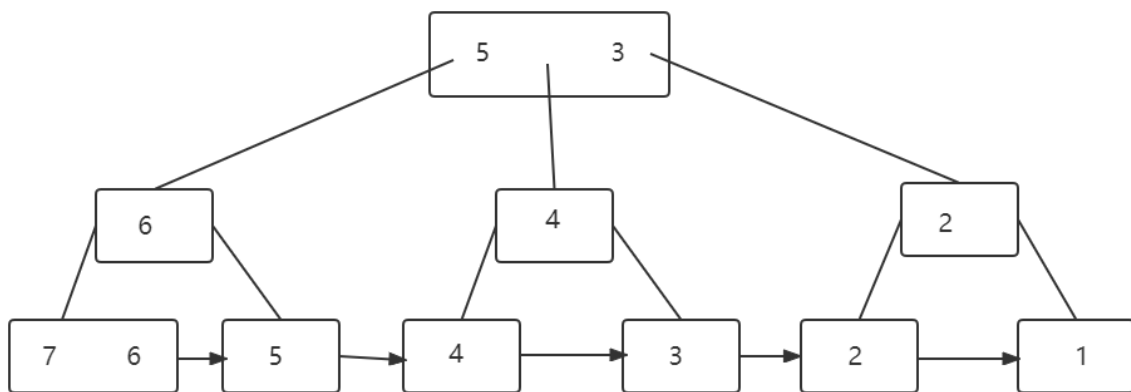
到此为止，大家应该对升序索引和降序索引有了一个大概的了解，但并没有真正理解，因为大家并不知道升序索引与降序索引底层到底是如何实现的。

降序索引的底层实现

升序索引对应的B+树



降序索引对应的B+树

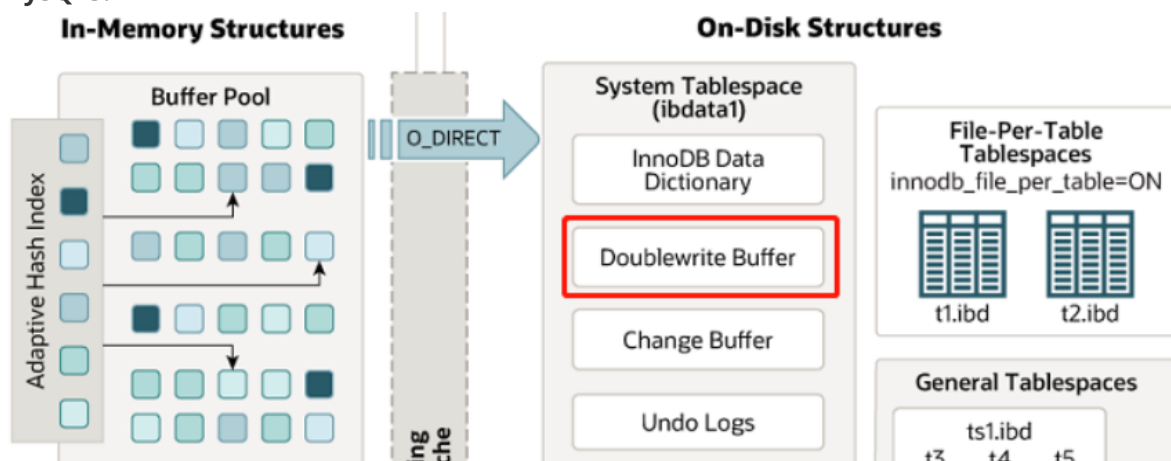


如果没有降序索引，查询的时候要实现降序的数据展示，那么就需要把原来默认是升序排序的数据处理一遍（比如利用压栈和出栈操作），而降序索引的话就不需要，所以在优化一些SQL的时候更加高效。

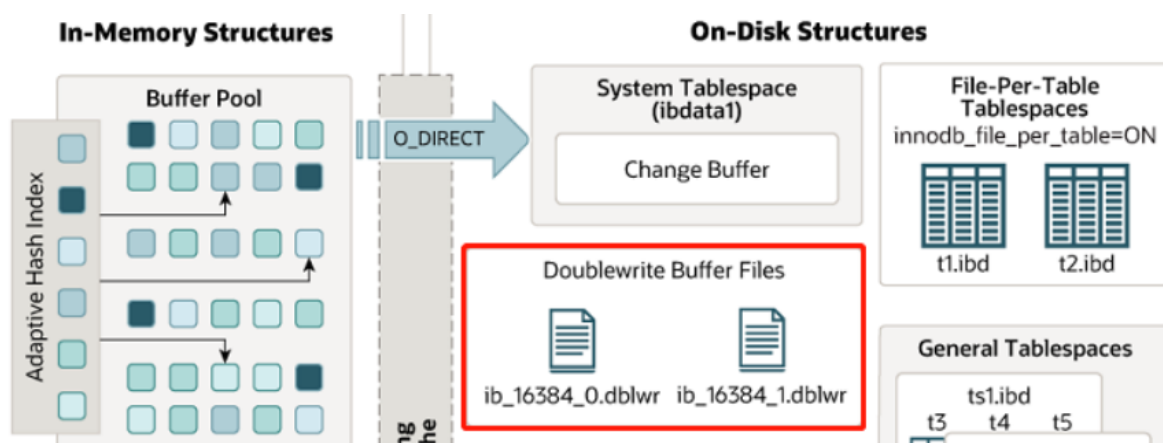
还有一点，现在 **只有Innodb存储引擎支持降序索引**。

Doublewrite Buffer的改进

MySQL5.7



MySQL8.0



在MySQL 8.0.20 版本之前，doublewrite 存储区位于系统表空间，从 8.0.20 版本开始，doublewrite 有自己独立的表空间文件，这种变更，能够降低doublewrite的写入延迟，增加吞吐量，为设置doublewrite文件的存放位置提供了更高的灵活性。

因为系统表空间在存储中就是一个文件，那么doublewrite必然会受制于这个文件的读写效率（其他向这个文件的读写操作，比如统计、监控等数据操作）

系统表空间(system tablespace)

这个所谓的系统表空间可以对应文件系统上一个或多个实际的文件，默认情况下，InnoDB会在数据目录下创建一个名为ibdata1(在你的数据目录下找找看有木有)、大小为12M的文件，这个文件就是对应的系统表空间在文件系统上的表示。

总用量 122972									
-rw-r-----.	1	mysql	mysql	56	5月	13	18:17	auto.cnf	
-rw-----.	1	mysql	mysql	1676	5月	13	18:17	ca-key.pem	
-rw-r-----.	1	mysql	mysql	1112	5月	13	18:17	ca.pem	
-rw-r-----.	1	mysql	mysql	18851	5月	13	22:47	centosvm-slow.log	
-rw-r--r--.	1	mysql	mysql	1112	5月	13	18:17	client-cert.pem	
-rw-----.	1	mysql	mysql	1676	5月	13	18:17	client-key.pem	
-rw-r-----.	1	mysql	mysql	393	5月	31	16:08	ib_buffer_pool	
-rw-r-----.	1	mysql	mysql	12582912	6月	10	18:07	ibdata1	
-rw-r-----.	1	mysql	mysql	50331648	6月	10	18:07	ib_logfile0	
-rw-r-----.	1	mysql	mysql	50331648	5月	13	18:17	ib_logfile1	
-rw-r-----.	1	mysql	mysql	12582912	6月	10	16:44	ibtmp1	
drwxr-x---	2	mysql	mysql	52	6月	10	18:07	lijin	
drwxr-x---	2	mysql	mysql	4096	5月	27	20:56	mysql	
srwxrwxrwx.	1	mysql	mysql	0	6月	10	16:44	mysql.sock	
-rw-----.	1	mysql	mysql	5	6月	10	16:44	mysql.sock.lock	
drwxr-x---	2	mysql	mysql	8192	5月	13	18:17	performance_schema	

而单独的文件必然效率比放在系统表空间效率要高!!!

新增的参数:

innodb_doublewrite_dir

指定doublewrite文件存放的目录，如果没有指定该参数，那么与innodb数据目录一致 (innodb_data_home_dir)，如果这个参数也没有指定，那么默认放在数据目录下面(datadir)。

innodb_doublewrite_files

指定doublewrite文件数量，默认情况下，每个buffer pool实例，对应2个doublewrite文件。

innodb_doublewrite_pages

一次批量写入的doublewrite页数量的最大值，默认值、最小值与innodb_write_io_threads参数值相同，最大值512。

innodb_doublewrite_batch_size

一次批量写入的页数量。默认值为0，取值范围0到256。

redo log 无锁优化

[MySQL :: MySQL 8.0: New Lock free, scalable WAL design](#)

MySQL 8 中快速添加列的底层实现原理

MySQL 8 中快速添加列的底层实现原理是通过 InnoDB 存储引擎的 "Fast Index Creation" 特性实现的。该特性允许在大型表中高效地添加列，而无需重建整个表。

当向表中添加新列时，MySQL 8 使用一种称为 "in-place ALTER" 的技术，以修改表的元数据，而无需进行完整的表重建。在添加新列的情况下，InnoDB 存储引擎会创建一个 "不可见" 的表的副本，该副本包含了新列。然后，这个副本会以非阻塞的方式与原始表进行同步，使得正常的操作可以继续进行，而不会中断。

这种 "in-place ALTER" 技术利用了 InnoDB 的高效数据存储格式，该格式使用聚簇索引来物理存储数据。通过将新列作为聚簇索引的一部分添加，MySQL 8 避免了复制和重新组织整个表的需要，从而实现了更快的列添加操作。

快速加列原理

快速加列语法

```
alter table table add  
column c1 int,  
algorithm=instant;
```

快速加列限制

- 只能在表的最后
- 不能添加主键列
- 不支持压缩的表格式

